

LEOPARDO RH

Design System — Dossier Complet

**Timing · Points oubliés · Décisions
architecturales · Règles d'or**

Parties : 4 — Timing · Oublis · Décisions architecturales · Règles d'or
Tâches : T-DS01 → T-DS30 (30 tâches design complètes)
Effort total : ~35h réparties sur 3 semaines · 2h/jour en parallèle
Décisions urgentes : décisions architecturales à prendre avant Sprint 5
Statut : Sprint 4 Béta — fenêtre design critique ouverte

TABLE DES MATIÈRES

PARTIE 1	Faut-il intégrer le design maintenant ?	Analyse honnête Sprint 4 — état réel, décision binaire
1.1	La question directe et la réponse	Tableau état actuel vs verdicts
1.2	Pourquoi le design AVANT la bêta	Règle produit SaaS B2B — crédibilité commerciale
1.3	Ordre d'exécution recommandé	Semaine par semaine avec tâches parallèles
PARTIE 2	Points oubliés — T-DS23 à T-DS30	8 tâches supplémentaires non couvertes initialement
T-DS23	SplashScreen + LaunchScreen natif	Flutter · Haute
T-DS24	Messages d'erreur API humanisés	Flutter · Critique
T-DS25	Skeleton screens	Flutter · Haute
T-DS26	BottomSheet résumé après check-out	Flutter · Moyenne
T-DS27	Vue liste employés Blade	Web Blade · Critique
T-DS28	Vue détail employé + pointages	Web Blade · Haute
T-DS29	Favicon + icône + métadonnées	Transversal · Haute
T-DS30	Accessibilité minimale WCAG AA	Transversal · Haute
PARTIE 3	Récapitulatif T-DS01 → T-DS30	Toutes les tâches · timing · efforts · bilan
PARTIE 4	Décisions architecturales design	3 décisions à prendre maintenant — impact modules futurs
4.1	Couleur comme langage de domaine	Vert = RH · Bleu = Sécurité · Ambre = Finance
4.2	Navigation pilotée par configuration	Sidebar dynamique · modules déclaratifs
4.3	BottomNav Flutter adaptative	Onglets calculés depuis les modules actifs du plan
PARTIE 5	Règles d'or design — Checklist PR	R1-R9 + contexte étendu — à intégrer dans PILOTAGE.md
PARTIE 6	Guide d'implémentation rapide	Code de référence pour les composants clés

1

FAUT-IL INTÉGRER LE DESIGN MAINTENANT ?

Analyse honnête · Sprint 4 · État réel du projet

Contexte : vous êtes en Sprint 4 — bêta. Le backend est déployé ou en cours de déploiement sur Render. Les tâches critiques T19-T34 ont été partiellement implémentées. La question du timing design est légitime et la réponse n'est pas binaire.

1.1 — La question directe et la réponse

Le tableau ci-dessous évalue chaque composant design selon son état actuel et l'impact réel qu'un report aurait sur la bêta commerciale :

Élément	État actuel	Impact si on reporte	Verdict
App Flutter	Dark only, Roboto, pas de BottomNav	Client bêta ouvre l'app → impression "app de développeur" → perd confiance dès le premier contact	FAIRE MAINTENANT
Dashboard Blade	Pas de sidebar, labels cassés, dark slate	Le manager évaluant le produit voit un dashboard sans navigation → ne comprend pas le produit	FAIRE MAINTENANT
LeopardoBadge (statuts)	Texte brut sans couleur	Les statuts sont l'information la plus lue du produit. Sans couleur, illisibles en 1 coup d'œil.	FAIRE MAINTENANT
Light mode Flutter	Inexistant	Amina (pharmacie, iPhone, soleil) : app illisible sur terrain. Premier retour bêta = rejet.	FAIRE MAINTENANT
Messages d'erreur API	Codes bruts (INVALID_CREDENTIALS)	Un utilisateur lambda ne comprend pas ces codes. Retours pollués.	FAIRE MAINTENANT
Animations avancées PulseButton	Correctes	Aucun impact bêta	REPORTER PHASE 2
Onboarding screens	Inexistants	Pas encore de self-service → reporter	REPORTER PHASE 2
Thème RTL (arabe)	Inexistant	Aucun client arabe en bêta — faible impact immédiat	REPORTER PHASE 3

Les 5 premières lignes marquées FAIRE MAINTENANT représentent environ 16h au total et peuvent être faites sur des branches feature parallèles sans bloquer le déploiement backend.

1.2 — Pourquoi le design AVANT la bêta et pas après

La logique intuitive est "d'abord la fonctionnalité, ensuite le design". C'est vrai pour un outil interne. Pour un SaaS B2B dont le prix est 29-199€/mois, c'est **faux** pour une raison simple : la première impression conditionne la décision d'achat. Vos 3-5 prospects bêta vont évaluer le produit EN MÊME TEMPS que vous collectez leurs retours.

Scénario	Ce que voit le prospect	Retour probable	Utilité du retour
Design actuel (dark, pas de sidebar)	App sombre illisible + dashboard sans navigation + erreurs en code	"C'est pas fini" ou "l'interface est compliquée" ou "je comprends pas les erreurs"	0% — pollué par le design, non par la fonctionnalité
Design refaits (T-DS01 à T-DS30 critiques)	App propre light/dark + sidebar navigation + badges colorés + messages clairs	"Le pointage est lent" ou "je veux voir les congés" ou "l'export manque"	100% — retours sur la vraie valeur produit

Règle produit absolue : ne jamais présenter un SaaS B2B à un prospect avec un design qui crie "prototype". Le design n'est pas de la cosmétique — c'est de la crédibilité commerciale. Un dashboard sans sidebar n'est pas "incomplet visuellement", il est incompréhensible fonctionnellement.

1.3 — Ordre d'exécution recommandé

Exécution en deux branches parallèles : **feature/design-flutter** (indépendante du backend) et **feature/design-blade** (indépendante du Flutter). Les deux peuvent être menées simultanément.

Semaine	Tâches design	Tâches infra (parallèle)	Résultat livrable
Cette semaine (Sem. 1)	T-DS01 à DS04 (Inter + AppTheme + ThemeMode + BottomNav) ~4h + T-DS24 (erreurs API) ~1h30 + T-DS13 à DS16 (Tailwind + sidebar + KPI + badge) ~3h45 + T-DS27 (employees/index Blade) ~2h + T-DS29 (favicon + méta) ~45min	Déploiement backend Render Configuration VPS staging	App avec light mode Navigation propre Dashboard avec sidebar Messages d'erreur humains
Semaine 2 (Sem. 2)	T-DS05 à DS08 (Login + LeopardoBadge + AttendanceScreen + HistoryScreen) ~7h30 + T-DS23 (SplashScreen) ~1h + T-DS25 (Skeleton screens) ~1h + T-DS12 (ShimmerLoading) ~30min + T-DS10 (MetricCard) ~45min + T-DS28 (détail employé + pointages) ~1h30	Mobile branché sur bêta réelle Invitations prospects bêta	App présentable pour les 3-5 prospects Dashboard avec liste employés
Fin Sprint 4 (Sem. 3)	T-DS09, DS11 (Settings + EmptyState Flutter) ~2h30 + T-DS17 à DS22 (login web + employés + alertes + labels) ~7h + T-DS26 (BottomSheet checkout) ~45min + T-DS30 (accessibilité WCAG) ~1h	Collecte retours bêta Itérations backend	Design complet avant premiers retours structurés

2

POINTS OUBLIÉS — T-DS23 À T-DS30

8 tâches supplémentaires non couvertes dans le document design initial

Ces 8 tâches ont été identifiées après relecture complète du code. Elles complètent les fondations T-DS01 à T-DS22 sans les remettre en cause. Les lignes marquées **CRITIQUE** sont à intégrer cette semaine.

T-DS23 SplashScreen + LaunchScreen natif

FLUTTER

Actuellement, l'app actuel affiche directement le router sans splash — l'app "flashe" blanc au démarrage. → Créer lib/features/auth/screens/splash_screen.dart avec le logo Leopardo sur fond navy (#0F2544), durée 1.2s puis redirection selon authState. → Dans GoRouter, définir /splash comme initialLocation. Le router redirige selon authState.isLoading. → iOS : configurer LaunchScreen.storyboard avec fond navy pour éviter le flash blanc natif. → Android : configurer styles.xml avec windowBackground navy.

HAUTE

T-DS24 Messages d'erreur API humanisés

FLUTTER

Actuellement, les erreurs s'affichent en codes bruts (ex: "INVALID_CREDENTIALS"). Un utilisateur lambda ne comprend pas. → Créer lib/core/api/error_messages.dart — map String errorCode → String message français humain. → INVALID_CREDENTIALS → "Email ou mot de passe incorrect." → ALREADY_CHECKED_IN → "Vous avez déjà pointé votre arrivée." → MISSING_CHECK_IN → "Pointez d'abord votre arrivée avant de sortir." → ACCOUNT_SUSPENDED → "Votre compte a été suspendu. Contactez votre responsable." → Dans api_client.dart, intercepter les erreurs et utiliser ce mapper avant de les propager. → SnackBar : SnackBarBehavior.floating, borderRadius 10, fond G800, texte blanc.

CRITIQUE

T-DS25 Skeleton screens (pas seulement shimmer)

FLUTTER

Actuellement, le ShimmerLoading existant est un rectangle générique. Un skeleton screen reproduit la structure réelle de la page. → Créer lib/core/widgets/skeletons/attendance_skeleton.dart — structure AttendanceScreen : cercle (PulseButton placeholder) + 2 rectangles (MetricCards). → Créer lib/core/widgets/skeletons/history_skeleton.dart — liste de lignes avec avatar rond + 2 rectangles de largeurs différentes. → Utiliser ShimmerLoading existant comme base pour les formes individuelles. → Remplacer les CircularProgressIndicator par les skeletons dans AttendanceScreen.

HAUTE

T-DS26 BottomSheet résumé après check-out

FLUTTER

Après un check-out réussi, l'app affiche seulement le PulseButton inversé. Il manque un résumé visuel immédiat. → Afficher un BottomSheet (non dismissable) pendant 3 secondes après check-out. → Contenu : icône check vert animé, heures travaillées (ex: 8h15), gain estimé, statut (ontime/retard). → Après 3 secondes ou tap du bouton "OK", le sheet se ferme et l'état se met à jour. → Utiliser les couleurs du LeopardoBadge pour le statut dans ce sheet.

MOYENNE

T-DS27 Vue liste employés Blade (employees/index)

WEB BLADE

Écrite en **CRITIQUE**
La vue employees/index n'existe pas dans les Blade actuels (seulement create et show). Il manque une page liste complète.
→ Créer resources/views/employees/index.blade.php — tableau paginé : avatar initiales coloré, nom complet, rôle (badge), statut (x-attendance-badge), actions (Voir / Modifier / Archiver). → WebEmployeeController::index() doit passer ->paginate(25) au lieu de get(). → Ajouter la route employees.index dans routes/web.php et dans la sidebar (nav-item "Collaborateurs"). → Barre de recherche simple (Alpine.js + filtre côté client par nom sur les 25 résultats chargés).

T-DS28 Vue détail employé + pointages + correction

WEB BLADE

Écrite en **HAUTE**
employees/show.blade.php existe mais ne montre pas les pointages. Il faut les intégrer. → Ajouter une section "Pointages récents" : tableau des 30 derniers logs avec x-attendance-badge, heures, check-in/out. → Bouton "Corriger un pointage" → x-modal avec formulaire PATCH /api/v1/attendance/{id} (tâche T19 du plan principal). → Cette page est le point d'entrée naturel du manager pour la correction — accessible depuis le tableau de présences.

T-DS29 Favicon + icône app + métadonnées web

TRANSVERSAL

Écrite en **HAUTE**
Le dashboard Blade n'a ni favicon ni balises meta. Quand le manager met la page en favori, il voit l'icône page blanche. → Créer un favicon SVG (carré vert #059669 avec "L" blanc en Helvetica-Bold) dans api/public/favicon.svg. → Générer favicon.ico avec ImageMagick : convert -background none favicon.svg -resize 32x32 favicon.ico → Dans layouts/app.blade.php : link rel=icon href=/favicon.svg, meta description "Gestion RH simplifiée pour PME", meta theme-color #0F2544. → Flutter : vérifier que l'icône Android (mipmap) correspond à l'identité visuelle navy/vert.

T-DS30 Accessibilité minimale WCAG AA

TRANSVERSAL

Écrite en **HAUTE**
Deux violations majeures identifiées dans le code actuel : → 1/ Bouton "Déconnexion" (TextButton rouge sur fond dark) — ratio contraste insuffisant. Correction : AppColors.red600 sur fond white (ratio 4.5:1 minimum). → 2/ Labels de formulaire en gray-300 (Blade) sur fond slate-950 — ratio insuffisant. Remplacer text-slate-300 par text-gray-700 sur fond blanc. → Flutter : tous les IconButton doivent avoir un Tooltip pour les lecteurs d'écran. → Blade : tous les inputs doivent avoir un associé (for + id). Vérifier que x-platform.input génère le bon id. → Test : Chrome DevTools > Accessibility pour valider les ratios.

3

RÉCAPITULATIF COMPLET T-DS01 → T-DS30

Toutes les tâches design · timing · efforts · bilan

ID	Titre	Surface	Effort	Priorité	Quand	Note
T-DS01	Intégrer la police Inter	FLUTTER	~30min	CRITIQUE	Cette semaine J1	Remplacer Roboto par Inter dans pubspec.yaml + AppTheme
T-DS02	Remplacer AppTheme complet	FLUTTER	~1h	CRITIQUE	Cette semaine J1	Palette navy/vert, TextTheme, InputDecoration, ElevatedButton
T-DS03	ThemeMode.system dans app.dart	FLUTTER	~15min	CRITIQUE	Cette semaine J1	MaterialApp.router(themeMode: ThemeMode.system)
T-DS04	BottomNavigationBar + AppShell	FLUTTER	~2h	CRITIQUE	Cette semaine J2	Shell route GoRouter + 4 onglets manager / 2 onglets employé
T-DS05	Refonte LoginScreen Flutter	FLUTTER	~2h	HAUTE	Semaine 2 J1	Logo centré, fond navy, champs Inter, bouton vert
T-DS06	Composant LeopardoBadge	FLUTTER	~1h	CRITIQUE	Semaine 2 J1	Enum AttendanceStatus + Container coloré + texte
T-DS07	Refonte AttendanceScreen	FLUTTER	~3h	HAUTE	Semaine 2 J2	PulseButton, MetricCards, statuts, dark+light
T-DS08	Refonte HistoryScreen	FLUTTER	~1h30	HAUTE	Semaine 2 J3	ListView, HistoryTile, badges, skeleton
T-DS09	Refonte SettingsScreen	FLUTTER	~2h	MOYENNE	Fin Sprint 4	Sections, logout rouge, langue, thème
T-DS10	Composant MetricCard Flutter	FLUTTER	~45min	HAUTE	Semaine 2 J2	title + value + icon + couleur + tendance
T-DS11	Composant EmptyState Flutter	FLUTTER	~30min	MOYENNE	Fin Sprint 4	Illustration + titre + message + CTA optionnel
T-DS12	ShimmerLoading light mode	FLUTTER	~30min	HAUTE	Semaine 2 J1	Gradient adaptatif light/dark mode
T-DS13	tailwind.config.js palette navy	WEB BLADE	~15min	CRITIQUE	Cette semaine J3	Étendre palette avec navy-50→950, vert Leopardo
T-DS14	Nouveau layout sidebar app.blade.php	WEB BLADE	~2h	CRITIQUE	Cette semaine J3	Sidebar fixe, nav-items, logo, profil bas, responsive

ID	Titre	Surface	Effort	Priorité	Quand	Note
T-DS15	Composant x-kpi-card	WEB BLADE	~45min	CRITIQUE	Cette semaine J3	@props title, value, delta, icon, color
T-DS16	Composant x-attendance-badge	WEB BLADE	~30min	CRITIQUE	Cette semaine J3	@props status — badge HTML coloré avec les mêmes couleurs que LeopardoBadge Flutter
T-DS17	Refonte page login web	WEB BLADE	~1h	HAUTE	Fin Sprint 4	Split layout, fond navy, form centré, fond image blur
T-DS18	Refonte page création employé	WEB BLADE	~2h	HAUTE	Fin Sprint 4	Sections groupées, validation inline, step indicator
T-DS19	Composant x-alert	WEB BLADE	~30min	HAUTE	Fin Sprint 4	success/warning/error/info avec icône et dismiss
T-DS20	Composant x-empty-state Blade	WEB BLADE	~30min	MOYENNE	Fin Sprint 4	SVG inline + message + CTA pour listes vides
T-DS21	Corriger tous les labels UI	WEB BLADE	~1h	HAUTE	Fin Sprint 4	grep employé/Biometrie/prenom → 0 résultat avec accents
T-DS22	Validation visuelle cross-surface	TRANSVERSAL	~1h	HAUTE	Fin Sprint 4	Vérifier cohérence couleurs statuts Flutter vs Blade
T-DS23	SplashScreen + LaunchScreen natif	FLUTTER	~1h	HAUTE	Semaine 2 J1	Logo sur fond navy, 1.2s, redirection authState
T-DS24	Messages d'erreur API humanisés	FLUTTER	~1h30	CRITIQUE	Cette semaine J2	error_messages.dart map codes → français lisible
T-DS25	Skeleton screens	FLUTTER	~1h	HAUTE	Semaine 2 J2	attendance_skeleton + history_skeleton
T-DS26	BottomSheet résumé après check-out	FLUTTER	~45min	MOYENNE	Fin Sprint 4	Résumé journée 3s + statut coloré
T-DS27	Vue liste employés Blade	WEB BLADE	~2h	CRITIQUE	Cette semaine J4	employees/index + paginate(25) + recherche Alpine
T-DS28	Vue détail employé + pointages	WEB BLADE	~1h30	HAUTE	Semaine 2 J4	Section pointages récents + modal correction
T-DS29	Favicon + icône + métadonnées web	TRANSVERSAL	~45min	HAUTE	Cette semaine J4	SVG favicon + meta description + theme-color
T-DS30	Accessibilité minimale WCAG AA	TRANSVERSAL	~1h	HAUTE	Fin Sprint 4	Contraste 4.5:1 + Tooltip + labels associés

Bilan des efforts par vague

Vague	Tâches incluses	Nb tâches	Effort estimé	Moment
Fondations Flutter critiques	T-DS01 à DS04 + DS24	5	~5h15	Cette semaine
Fondations web + oublis critiques	T-DS13 à DS16 + DS27 + DS29	6	~7h15	Cette semaine
Écrans Flutter complets	T-DS05 à DS12 + DS23 + DS25	12	~13h45	Semaine suivante
Écrans Blade complets	T-DS17 à DS22 + DS28	7	~8h	Semaine suivante
Finitions cross-surface	T-DS26 + DS30 + DS22	3	~2h45	Fin Sprint 4
TOTAL T-DS01 à T-DS30	Toutes les tâches design	30	~35h	3 semaines

35h réparties sur 3 semaines = environ 2h par jour en parallèle des tâches backend. Les branches feature/design-flutter et feature/design-blade sont totalement indépendantes et peuvent être menées simultanément.

4

DÉCISIONS ARCHITECTURALES DESIGN

3 décisions à prendre maintenant — impact direct sur les modules futurs

Avec les modules *Caméras (Phase 1 Prime)*, *Alarmes* et *Finance* planifiés, *Leopardo RH* passe d'un outil RH à une plateforme de gestion opérationnelle PME. Ce glissement de positionnement a des conséquences directes sur le design — 3 décisions doivent être prises maintenant, avant de construire d'autres modules.

Décision 1 — La couleur comme langage de domaine

Actuellement tout est vert — cohérent pour un outil mono-domaine. Dès qu'on ajoute un module sécurité (caméras, alarmes) et un module finance, le vert ne peut plus signifier "actif" en général, il doit signifier "RH". Changer cette règle après avoir construit trois modules, c'est une refonte complète de la charte couleur.



Application concrète — règle à écrire dans *AppColors* et dans *tailwind.config.js* :

```
// lib/core/theme/app_colors.dart
class DomainColors {
  // RH — vert Leopardo
  static const rh = Color(0xFF059669); // primary
  static const rhLight = Color(0xFFD1FAE5); // backgrounds
  static const rhDark = Color(0xFF065F46); // text on light
  // Sécurité — bleu
  static const security = Color(0xFF2563EB);
  static const securityLight = Color(0xFFDBEAFE);
  static const securityDark = Color(0xFF1E40AF);
  // Finance — ambre
  static const finance = Color(0xFFD97706);
  static const financeLight = Color(0xFFFFEF3C7);
  static const financeDark = Color(0xFF92400E);
}
```

1 Couleur par domaine

Si on le fait MAINTENANT

La couleur encode immédiatement le module. Un badge vert = RH. Un badge bleu = sécurité. Un manager reconnaît le contexte en 0,2 secondes sans lire le texte.

Si on le reporte

Il faut changer la signification du vert dans 3 modules déjà construits. LeopardoBadge et x-attendance-badge devront être refactorisés. Le chantier prend 2-3 jours au lieu de 2 heures maintenant.

Coût du report

Refactoring de tous les composants de statut existants. Confusion temporaire pour les utilisateurs lors de la transition. Risque de régression visuelle.

Décision 2 — Navigation pilotée par configuration

La sidebar app.blade.php actuelle a les liens écrits en dur. Avec 8-10 modules possibles dont chacun dépend du plan souscrit (Starter/Business/Enterprise), cette approche devient ingérable en maintenance et génère des bugs subtils (lien visible pour un plan qui n'y a pas accès).

```
// config/navigation.php – à créer
return [
    'modules' => [
        ['id'=>'rh',      'label'=>'RH & Pointage', 'icon'=>'users', 'route'=>'dashboard.index',
         'color'=>'green', 'min_plan'=>'starter',  'domain'=>'rh'],
        ['id'=>'cameras', 'label'=>'Surveillance', 'icon'=>'camera', 'route'=>'cameras.index',
         'color'=>'blue',  'min_plan'=>'business', 'domain'=>'security'],
        ['id'=>'finance',  'label'=>'Finance',      'icon'=>'wallet', 'route'=>'finance.index',
         'color'=>'amber', 'min_plan'=>'enterprise', 'domain'=>'finance'],
        ['id'=>'alarms',   'label'=>'Alarmes',      'icon'=>'bell',   'route'=>'alarms.index',
         'color'=>'red',   'min_plan'=>'business',   'domain'=>'security'],
    ]
];

// Dans app.blade.php – la sidebar se génère depuis ce tableau :
@foreach(config('navigation.modules') as $module)
    @if(auth()->user()->company->plan->hasModule($module['id']))
        <x-nav-item :module="$module" />
    @endif
@endforeach
```

2 Sidebar dynamique

Si on le fait MAINTENANT

Ajouter un nouveau module = ajouter 1 entrée dans le tableau de config. La sidebar se met à jour automatiquement. L'accès est filtré par plan sans aucun if/else dans les vues. Chaque module est autonome.

Si on le reporte

Chaque nouveau module nécessite de modifier app.blade.php. Risque d'oubli de vérification du plan. Duplication des conditions d'accès entre le backend (middleware) et le frontend (sidebar). Impossibilité de reconfigurer sans déploiement.

Coût du report

Modification manuelle de la sidebar à chaque module (1-2h par module × 8 modules = 16h). Bugs de sidebar visible/invisible incorrects. Refactoring complet quand le nombre de modules dépasse 5.

Décision 3 — BottomNavigationBar Flutter adaptative

Le BottomNavigationBar de T-DS04 ne doit pas avoir 4 onglets fixes. Il doit calculer ses onglets depuis les modules actifs du plan de l'entreprise. Un manager Plan Business + Caméras devrait avoir 4 onglets. Avec Enterprise + Finance + Caméras + Alarmes, il en a 5. Construire ça de façon rigide maintenant et le refactorer à la Phase 3 coûte 2x plus cher.

```
// lib/core/navigation/app_shell.dart
class AppShell extends StatelessWidget {
  const AppShell({super.key, required this.child});
  final Widget child;

  @override
  Widget build(BuildContext context) {
    final plan = context.watch<AuthBloc>().state.company.plan;
    final tabs = ModuleRegistry.tabsForPlan(plan); // List<AppTab>

    return Scaffold(
      body: child,
      bottomNavigationBar: NavigationBar(
        destinations: tabs.map((t) => NavigationDestination(
          icon: Icon(t.icon, color: t.domainColor),
          label: t.label,
        )).toList(),
      ),
    );
  }
}

// lib/core/navigation/module_registry.dart
class ModuleRegistry {
  static List<AppTab> tabsForPlan(Plan plan) => _allTabs
    .where((t) => plan.hasModule(t.moduleId))
    .toList();
  static const _allTabs = [
    AppTab(id: 'rh', label: 'Accueil', icon: Icons.home, moduleId: 'rh', domain: Domain.rh),
    AppTab(id: 'cameras', label: 'Caméras', icon: Icons.videocam, moduleId: 'cameras', domain: Domain.security),
    AppTab(id: 'finance', label: 'Finance', icon: Icons.account_balance, moduleId: 'finance', domain: Domain.finance),
    AppTab(id: 'alarms', label: 'Alarmes', icon: Icons.notifications, moduleId: 'alarms', domain: Domain.security),
    AppTab(id: 'settings', label: 'Paramètres', icon: Icons.settings, moduleId: 'core', domain: Domain.system),
  ];
}
```

3 BottomNav adaptative

Si on le fait MAINTENANT

Ajouter un module = ajouter 1 AppTab dans ModuleRegistry. L'onglet apparaît automatiquement pour les plans concernés. Le design s'adapte au plan sans code conditionnel dans les widgets. Testable unitairement.

Si on le reporte

4 onglets fixes dans le BottomNavigationBar. Quand les caméras arrivent, il faut modifier AppShell en dur. Quand la finance arrive, re-modifier. Impossibilité de configurer par plan sans if/else dans chaque widget.

Coût du report

Refactoring complet de AppShell et des routes GoRouter à chaque nouveau module (4-8h × 4 modules = 32h). Risque de casser la navigation existante. Tests à reprendre depuis zéro.

Résumé des 3 décisions : (1) Couleur par domaine — à appliquer maintenant dans AppColors et tailwind.config.js avant tout nouveau composant. (2) Sidebar dynamique — à implémenter pendant T-DS14 au lieu d'écrire les liens en dur. (3) BottomNav adaptative — à implémenter pendant T-DS04 au lieu de mettre 4 onglets fixes. Ces 3 décisions représentent +2h de travail maintenant contre +50h de refactoring plus tard.

5

RÈGLES D'OR DESIGN — CHECKLIST PR

R1-R9 + contexte étendu — à intégrer dans PILOTAGE.md sous «10 RÈGLES ABSOLUES»

Ces 9 règles s'appliquent à **TOUTE** modification de code UI, qu'elle soit faite manuellement ou par une session IA. Elles ont le même statut que les règles backend (tenant isolation, horodatage serveur, etc.). Aucune PR touchant l'UI ne doit être mergée sans que ces règles aient été vérifiées.

R1

Pas de couleur hardcodée

FLUTTER

Color(0xFF...) ou Colors.xxx sont interdits dans un widget. Utiliser toujours AppColors.xxx ou DomainColors.xxx. Les seules exceptions tolérées sont les couleurs de debug (affichées seulement en assert) et les couleurs générées dynamiquement depuis une API.

R2

Pas de style inline Blade

WEB

style="color:#..." ou style="background:..." sont interdits dans une vue Blade. Utiliser toujours des classes Tailwind configurées dans tailwind.config.js. Les seules exceptions sont les attributs SVG fill/stroke.

R3

Statuts = composants dédiés

LES DEUX

Tout statut RH (présent, absent, retard, congé, suspendu) passe par LeopardoBadge (Flutter) ou x-attendance-badge (Blade). Aucun texte brut, aucun Container ad hoc, aucun Chip inline. Cette règle garantit la cohérence cross-surface.

R4

Couleurs des boutons normées

LES DEUX

CTA primaire → vert #059669 (DomainColors.rh). Action destructive → rouge #DC2626. Action secondaire / outline → outline NAVY #0F2544. Lien texte → bleu #2563EB. Bouton désactivé → gris #9CA3AF. Pas d'exceptions sans validation design.

R5

Français avec accents partout

LES DEUX

La commande `grep -r "employe\|Biometrie\|Creer\|prenom\|desactiv" resources/views/ lib/` doit retourner 0 résultat. Les accents sont obligatoires : employé, créer, désactiver, prénom, biométrie. Vérifier avant chaque commit.

R6

Light + dark testés à chaque PR Flutter

FLUTTER

Un widget invisible, illisible ou mal contrasté en light mode est un bug bloquant — aussi grave qu'une `NullPointerException`. Test obligatoire sur simulateur iPhone en light mode avant chaque PR. Le dark mode seul n'est pas suffisant.

R7

Cohérence cross-surface des couleurs de statut

LES DEUX

Si la couleur du statut "retard" change en Flutter (ex: amber → orange), elle doit changer dans la même PR dans x-attendance-badge Blade. Les deux surfaces doivent toujours être synchrones. Documenter le changement dans CHANGELOG.

R8

Contraste WCAG AA minimum
LES DEUX

Ratio texte/fond $\geq 4.5:1$ pour le texte normal, $\geq 3:1$ pour le grand texte (18pt+). Vérifier avec Chrome DevTools > Accessibility avant chaque merge touchant des couleurs. Outil rapide : contrast-ratio.com. Aucune dérogation.

R9

Jamais de texte d'erreur API brut affiché
FLUTTER

Les codes `INVALID_CREDENTIALS`, `MISSING_CHECK_IN`, `ACCOUNT_SUSPENDED` etc. ne doivent jamais atteindre l'interface utilisateur. Ils passent obligatoirement par `error_messages.dart` (T-DS24). Si un nouveau code d'erreur est ajouté à l'API, sa traduction française doit être ajoutée en même temps.

Action requise : copier ces 9 règles dans PILOTAGE.md dans la section «10 RÈGLES ABSOLUES» — en dessous des règles backend existantes. Les afficher aussi dans le README des dossiers /mobile et /api/resources pour qu'elles soient vérifiées à chaque session de développement.

6

GUIDE D'IMPLÉMENTATION RAPIDE

Code de référence pour les composants clés — copier-coller ready

AppColors et DomainColors (lib/core/theme/app_colors.dart)

```
import 'package:flutter/material.dart';

class AppColors {
  // Navy – fond principal, texte dark
  static const navy900 = Color(0xFF0A1B36);
  static const navy800 = Color(0xFF0F2544); // fond app dark mode
  static const navy700 = Color(0xFF1B3A6B);
  static const navy100 = Color(0xFFDBEAFE);
  static const navy50 = Color(0xFFEFF6FF);
  // Vert RH – actions primaires, statuts RH
  static const green600 = Color(0xFF059669); // CTA primaire
  static const green100 = Color(0xFFD1FAE5); // bg badge présent
  static const green800 = Color(0xFF065F46); // texte sur vert clair
  // Amber – avertissement, retard
  static const amber600 = Color(0xFFD97706);
  static const amber100 = Color(0xFFFFEF3C7);
  static const amber800 = Color(0xFF92400E);
  // Rouge – erreur, absent, destructif
  static const red600 = Color(0xFFDC2626);
  static const red100 = Color(0xFFFFEE2E2);
  static const red800 = Color(0xFF991B1B);
}

class DomainColors {
  static Color primary(String domain) => {
    'rh': AppColors.green600,
    'security': const Color(0xFF2563EB),
    'finance': AppColors.amber600,
    'system': AppColors.navy700,
  }[domain] ?? AppColors.navy700;
}
```

LeopardoBadge (lib/core/widgets/leopardo_badge.dart)

```
enum AttendanceStatus { present, absent, late, leave, pending }

class LeopardoBadge extends StatelessWidget {
  const LeopardoBadge({super.key, required this.status});
  final AttendanceStatus status;

  static const _labels = {
    AttendanceStatus.present: 'Présent', AttendanceStatus.absent: 'Absent',
    AttendanceStatus.late: 'En retard', AttendanceStatus.leave: 'Congé',
    AttendanceStatus.pending: 'En attente',
  };
  static const _bgs = {
    AttendanceStatus.present: AppColors.green100, AttendanceStatus.absent: AppColors.red100,
    AttendanceStatus.late: AppColors.amber100, AttendanceStatus.leave: AppColors.navy100,
    AttendanceStatus.pending: Color(0xFF3F4F6),
  };
  static const _fgs = {
    AttendanceStatus.present: AppColors.green800, AttendanceStatus.absent: AppColors.red800,
    AttendanceStatus.late: AppColors.amber800, AttendanceStatus.leave: AppColors.navy700,
    AttendanceStatus.pending: Color(0xFF374151),
  };
  @override
  Widget build(BuildContext context) => Container(
    padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 4),
    decoration: BoxDecoration(color: _bgs[status], borderRadius: BorderRadius.circular(20)),
    child: Text(_labels[status]!, style: TextStyle(
      color: _fgs[status], fontSize: 12, fontWeight: FontWeight.w600)),
  );
}
```

error_messages.dart (lib/core/api/error_messages.dart)

```
class ErrorMessages {
  static const _map = <String, String>{
    'INVALID_CREDENTIALS': 'Email ou mot de passe incorrect.',
    'ALREADY_CHECKED_IN': 'Vous avez déjà pointé votre arrivée.',
    'MISSING_CHECK_IN': 'Pointez d'abord votre arrivée avant de sortir.',
    'ACCOUNT_SUSPENDED': 'Votre compte a été suspendu. Contactez votre responsable.',
    'EMPLOYEE_NOT_ACTIVE': 'Votre compte est inactif. Contactez votre responsable.',
    'COMPANY_NOT_FOUND': 'Entreprise introuvable. Vérifiez votre email.',
    'TOKEN_EXPIRED': 'Votre session a expiré. Veuillez vous reconnecter.',
    'TOO_MANY_ATTEMPTS': 'Trop de tentatives. Réessayez dans 1 minute.',
  };

  static String fromCode(String? code) =>
    _map[code] ?? 'Une erreur est survenue. Veuillez réessayer.';
}

// Usage dans api_client.dart DioInterceptor :
// on DioException : message = ErrorMessages.fromCode(response?.data['error_code']);
```

x-attendance-badge (resources/views/components/attendance-badge.blade.php)

```

{{-- @props(['status']) --}}
@php
$config = [
    'present' => ['label'=>'Present', 'bg'=>'bg-green-100', 'text'=>'text-green-800'],
    'absent' => ['label'=>'Absent', 'bg'=>'bg-red-100', 'text'=>'text-red-800'],
    'late' => ['label'=>'En retard', 'bg'=>'bg-amber-100', 'text'=>'text-amber-800'],
    'leave' => ['label'=>'Conge', 'bg'=>'bg-blue-100', 'text'=>'text-blue-800'],
    'pending' => ['label'=>'En attente', 'bg'=>'bg-gray-100', 'text'=>'text-gray-700'],
][$status] ?? ['label'=>$status, 'bg'=>'bg-gray-100', 'text'=>'text-gray-700'];
@endphp
<span class="inline-flex items-center px-2.5 py-0.5 rounded-full text-xs font-semibold
        {{ $config['bg'] }} {{ $config['text'] }}">
    {{ $config['label'] }}
</span>

```

Sidebar dynamique app.blade.php (extrait)

```

{{-- resources/views/layouts/app.blade.php -- extrait sidebar --}}
<nav class="flex flex-col gap-1 px-3 py-4">
    @foreach(config('navigation.modules') as $module)
        @if(auth()->user()->company->plan->hasModule($module['id']))
            <x-nav-item
                :href="route($module['route'])"
                :label="$module['label']"
                :icon="$module['icon']"
                :domain-color="$module['color']"
                :active="request()->routeIs($module['route'].'*')"
            />
        @endif
    @endforeach
</nav>

{{-- x-nav-item component (resources/views/components/nav-item.blade.php) --}}
@props(['href','label','icon','domainColor'=>'green','active'=>false])
<a href="{{ $href }}"
    class="flex items-center gap-3 px-3 py-2 rounded-lg text-sm font-medium
        transition-colors {{ $active ? 'bg-navy-800 text-white' : 'text-gray-400 hover:bg-navy-800/50 hover:text-white' }}"
    <x-dynamic-component :component="'heroicon-o-'. $icon" class="w-5 h-5
        text-{{ $domainColor }}-400" />
    {{ $label }}
</a>

```

Ce document est la référence design complète pour Leopardo RH Sprint 4 et au-delà. Toute décision UI non couverte ici doit être documentée et ajoutée avant d'être implémentée. Version 2.0 — Avril 2026.